Batcher: One Engine for Every Scale, Every Shape of Data, and Every AI Workload

A Mergeable Algebra that Unifies Batch and Streaming, from a Core to a Cluster

Stephen Offer

Anyscale • stephen.offer@anyscale.com

Abstract

The data stack most companies run was designed between 2009 and 2014, on assumptions that were all true then and have all since inverted. A single instance now carries 96 to 192 cores and terabytes of memory. Change data capture removed the batch window. Object storage and schema drift removed the catalog, and the operators that replaced it, regexes, JSON paths, Python callbacks and models, carry no statistics at all. Machine-generated documents, images, audio and video became most of the corpus, and a model stopped consuming the finished dataset and started producing intermediate columns inside it. A pipeline is now I/O, then CPU decode, then GPU inference, then a CPU join, and it is written in Python.

The engines did not move with any of it, so a team is asked the same question over and over. Pick a scale, pick batch or streaming, pick a shape of data, pick a tier of model, pick a device, pick a language: every answer is a different product, with glue in between.

Batcher is an engine designed after all of them, and one constraint does most of the answering. A stateful operator may exist only once, as a mergeable triple partial $\rightarrow$ combine $\rightarrow$ finalize in Rust over Apache Arrow, so one core, ninety-six cores and a cluster differ in scheduling and in nothing else: small queries pay nothing for distribution, and large ones need no rewrite. That same triple *is* the incremental form, so a finite table is a stream that ends and both run under one memory bound. Decode, embedding, vector search and inference are expressions over those same Arrow batches under the same optimizer, across 30 file and table formats and 125 ML entry points. And because the operator is identical everywhere, a measurement taken anywhere is valid everywhere, which is what lets Batcher plan the next query from what it measured rather than from vendor constants.

The result is breadth that costs nothing. On relational work, one 96-core machine matches DuckDB on TPC-H [33] sf1, leads Polars and Daft by 1.8–2.2$\times$, sweeps PyArrow's Acero engine, and wins 43 of 43 ClickBench queries against DuckDB on identical input. On eight nodes, aggregate shuffle throughput rises superlinearly, 7.6$\times$ for 4$\times$ the hardware. On the AI pipelines a purpose-built runtime was designed for, it holds 2–3.6$\times$ against Ray Data while treating decode and inference as relational operators. And streaming inherits the batch operators unchanged: peak memory on a join-heavy query is 3.4 MB at one million rows and 3.3 MB at four million, against 198 MB and 794 MB for Batcher's own materializing executor on the same query. One axis is not yet won: reading raw Arrow, Batcher trails DuckDB's compressed native store with zone maps, and the evaluation isolates that gap to storage and not to execution.

1 Seven shifts since Spark

Every architecture encodes the world it was born into. The distributed data stack most companies still run was designed between 2009 and 2014, and it was designed well: the assumptions underneath it were all true at the time. Seven have since stopped being true, and none of them gradually. Each flipped hard enough to invert the design decision it justified. Six pillars answer the seven, because shifts four and five are two halves of one problem.

Shift one: the machine inverted. In 2012 a large server had 16 cores and 64 GB, so "the data will not fit on one machine" was simply a fact, and everything followed from it: a cluster you start before you ask a question, a scheduler between you and your data, serialization on every hop. A single cloud instance now has 96 to 192 cores and up to twelve terabytes of memory, but the tax stayed anyway, so a query that would take twelve milliseconds in-process still takes seconds through a distributed runtime. *Distribution stopped being a property of the engine and became a property of the query* (Section 2).

Shift two: data stopped arriving in batches. Change data capture, event logs and continuously landing object storage removed the nightly window that "daily freshness" was built around. The engines of the era answered by bolting a second execution model beside the first, which leaves two sets of operator semantics to hold in agreement and a rewrite for any pipeline that crosses between them. *Continuous arrival stopped being a special case and became the default* (Section 3).

Shift three: neither the data nor the transformations can be known before the query runs. Classical optimizers assumed curated schemas, statistics refreshed on a schedule, and queries written once and run for years. That world had `ANALYZE` jobs and DBAs; this one has object storage, schema drift and JSON columns. Worse, the operators themselves became opaque: a regular expression, a JSON extraction, a Python callback, an embedding model, an LLM call, none of which has a catalog entry, a selectivity, or a cost anybody has measured. A filter assumed to keep a tenth of its rows may

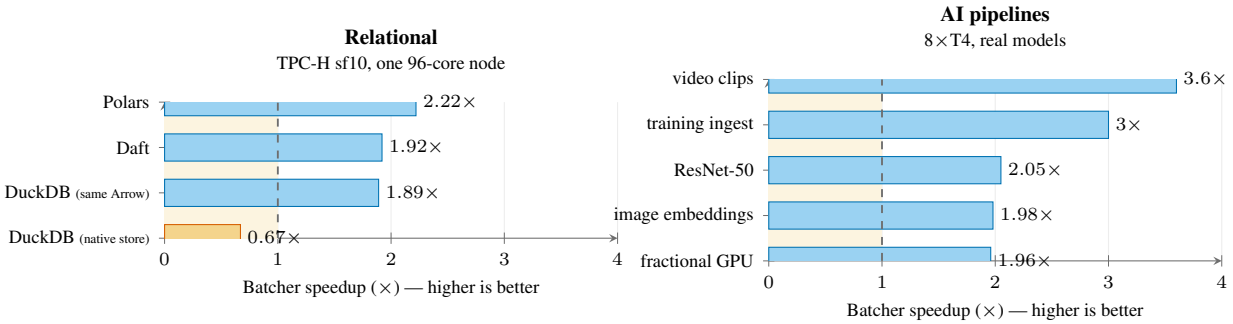

Figure 1: The claim in one picture: Batcher is not a relational engine that tolerates AI, nor an AI runtime that tolerates SQL. On the left it beats the fastest native engines on a standard SQL benchmark; on the right it beats a runtime purpose-built for GPU pipelines, on that runtime's own workloads, while treating decode and inference as relational operators under the same optimizer. The amber bar is the honest one: against DuckDB's compressed native store with zone maps, Batcher reading raw Arrow loses, and Section 13 isolates that to storage and not to kernels. Two axes not shown here move the same way: aggregate shuffle throughput rises 7.6× for 4× the nodes, and a join-heavy streaming query holds 3.4 MB where a materializing executor takes 198 MB.

keep all of them, and a model call priced like a comparison is off by six orders of magnitude. *Optimizing once, from numbers nobody measured, stopped being a performance choice and became a guess* (Section 4).

Shift four: the data became unstructured, and far larger. Analyst projections put the unstructured fraction of enterprise data at 80 to 90 percent [13, 27], a forecast and not a census. The share is the least interesting part of it: documents, logs, images, audio, video and sensor traces are produced continuously by machines, not periodically by people, so volume and rate rose together, and none of it arrives in columns. An engine whose type system stops at scalars can only treat this as an opaque blob to hand somewhere else. *Most of the data stopped being representable in the engine that was supposed to process it* (Section 5).

Shift five: AI moved from the end of the pipeline into the middle of it. This is the shift we think is most often missed. When machine learning consumed a finished dataset, keeping it outside the engine cost only a file handoff. It no longer consumes the output; it *produces* intermediate columns, and embedding a document or extracting fields with an LLM is a transformation with a join or a filter on both sides of it. Variety compounds it, because one team now runs generalized linear models, gradient-boosted trees, deep vision models and LLMs against the same tables, and each tier used to be a separate framework. *A model stopped being a consumer of the pipeline and became an operator inside it* (Section 5).

Shift six: compute stopped being homogeneous. The 2010s engine scheduled one resource, and a core was a core. A modern pipeline reads compressed bytes, decodes them on CPU, runs a model on GPU, and joins the result back on CPU, so the units are no longer interchangeable and the expensive one idles whenever the cheap one falls behind. *Utilization stopped being about parallelism and became about overlap across unlike devices* (Section 6).

Shift seven: the interface moved from SQL to Python. SQL did not go away and should not, but the practitioner is a Python programmer, the surrounding code is Python, and the model is a Python object. Engines answered with a user-defined-function escape hatch, which the optimizer cannot see into and which usually runs a row at a time in an interpreter, turning the natural way to express a transformation into the slowest way to run one. *The default authoring language stopped being the one the engine optimizes* (Section 7).

Each shift dissolved a boundary that used to be an architectural seam and is now a line one job routinely crosses: single-node and distributed, batch and streaming, structured and unstructured, query and model, CPU and GPU, SQL and Python. An engine built around either side of any of them must treat the other as foreign, and the bill arrives as glue. Six pillars pay it off, in order: one algebra spans a core to a cluster (Section 2), one set of operators serves batch and streaming (Section 3), one optimizer plans from measurement (Section 4), one expression surface covers every data shape and model tier (Section 5), one scheduler spans CPU and GPU (Section 6), and one Python surface stays transparent to the optimizer (Section 7).

The reason to care is Figure 1. Breadth of this kind is normally bought by being second-best everywhere, and it is not bought that way here: on a standard SQL benchmark Batcher leads the fastest native engines, and on GPU pipelines it leads the runtime built for them, in the same binary, from the same plan.

2 Pillar one: one algebra, every scale

Shift one: the machine inverted, so distribution should be a decision and not an architecture. An engine that assumes the cluster charges every small query for it; an engine that assumes the box cannot answer a large one. Both are now wrong often enough that neither is a safe default, so the choice has to move out of the engine and into the query.

The obstacle is easy to state. Ask an engine builder how a `GROUP BY` works and you get two answers: on one machine a hash table, on a cluster a shuffle, a partial pass and a reduce. The two share a name, a SQL semantics, and not a line of code, which means two performance profiles, two sets of bugs, and eventually two different answers. A float key that hashes one way locally and another after a shuffle splits one group in two, and the single-node test suite stays green throughout, because

the single-node implementation is correct.

Batcher makes that split impossible. A stateful operator may exist only as a mergeable triple, and the three execution regimes vary only its scheduling (Figure 2): one core folds a stream of batches, many cores partition and merge, and many machines partition, run the partial pass per node, ship pre-aggregated state, and combine in a tree. Formally, for any partitioning $\{p_k\}$ of an input P,

$$\mathsf{combine\_finalize}\big(\bigcup_k \mathsf{partial}(p_k)\big) = \mathsf{finalize}(\mathsf{partial}(P)), \tag{1}$$

which is a property test and not a comment: a generator produces random partitionings and asserts equality against the single-pass result, and a separate test asserts that a distributed run and a single-node run of the same plan return identical rows.

The paper's thesis follows from that. **There is no distribution tax on the small case**, because distribution is a scheduling decision made per query rather than a runtime you start. **And there is no rewrite on the way out**, because the operator that wins on one core is the operator that runs on eight nodes. Since mappers publish state that is already reduced, never raw, per-node memory stays bounded however wide the cluster grows, so shuffle throughput rises with node count instead of collapsing into contention (Section 13.5).

The triple itself is old: Gray et al.'s algebraic aggregate [12] and MapReduce's combiner [7]. Every distributed engine reaches for it somewhere, usually for aggregation. What differs here is treating it as an *admission criterion* for every stateful operator, join build, window and top-N included, so no second implementation ever exists to drift from the first. The cost is real and we pay it: an algorithm with no mergeable form does not get added.

3 Pillar two: batch and streaming are one execution model

Shift two: arrival became continuous, so a streaming query should not be a different program. A second execution model costs in three places: every operator is implemented and maintained twice, the two copies have to be proven to agree, and any pipeline that crosses between them has to be rewritten. That is every pipeline that succeeds, because freshness requirements only ever tighten.

The constraint that settles the scale split settles this one too. Batcher's executor is a streaming executor for *every* query. Operators pull small batches through a pipeline and materialize only at breakers, so peak memory is the breakers' state plus one batch per worker: a constant, independent of how much data arrives. On a join-heavy shape that is 3.4 MB at one million rows and 3.3 MB at four million, where Batcher's own materializing executor grows from 198 MB to 794 MB on the same query. The bounded case is not a special mode. A finite table is simply a stream that ends.

That works because the mergeable triple *is* the incremental form. An aggregate that folds each batch through partial and combine never reads its whole input, which is exactly what a continuous query needs: state that updates per arrival and finalizes on demand. The operator a batch query uses and the operator a continuous query needs are not two implementations that happen to agree. They are one implementation, and the difference is whether anything ever calls finalize. The same property makes an operator a pipeline breaker for *ordering* without making it a materialization point for *memory*, which is what gives a streaming aggregation over an unbounded source a memory bound at all.

So the memory story does not change with the source. A query over a hundred terabytes of Parquet and the same query over a Kafka topic run the same operators under the same bound, and a pipeline that outgrows its batch window does not get rewritten to survive.

4 Pillar three: an engine that learns

Shift three: neither the data nor the operators can be priced before the query runs. An engine facing that can keep inventing numbers, which is what a static cost model does, or it can measure itself and plan from what it saw. Everything in this section follows from taking the second option seriously, including the parts that look like over-engineering: if the plan is going to be wrong, the engine needs to find out *during* the query rather than after it.

Measuring itself is only sound because of the first pillar. Because the same operator runs everywhere, **a measurement taken anywhere is valid everywhere**: a cardinality observed on one core is evidence about the same operator on ninety-six. Refusing to fork the operator is what licenses the engine to plan from its own history. Every other engine in Table 1 ships its cost model as constants, chooses join strategy by heuristic, and sizes memory from a plan estimate, and those constants were tuned once, on hardware nobody here is running. Batcher replaces each with a measurement.

A single rule keeps this safe. **Every learned value is result-invariant.** Each bandit arm emits the same relation, a threshold picks between equivalent algorithms, a partition count only shards. Learning changes how much memory a query reserves, when it spills, how wide a batch is, and which of several equivalent algorithms runs. It can never change the answer. That is property-tested, and a cold store reproduces the untuned engine byte for byte, so the worst a stale value can do is cost throughput. This is a deliberately different bet from learned optimizers that predict plans [17, 21, 22], which change which plan runs in ways that are hard to bound. A weaker signal buys a far stronger safety property, and that is what lets the feature ship on by default.

4.1 Three clocks, not one

Adaptive query processing usually means one thing: re-plan at a stage boundary. Batcher does that too. It is the least interesting of the three places it adapts.

Microseconds: the filter reorders itself. A conjunctive filter evaluates its cheapest predicate first, which is what a static cost

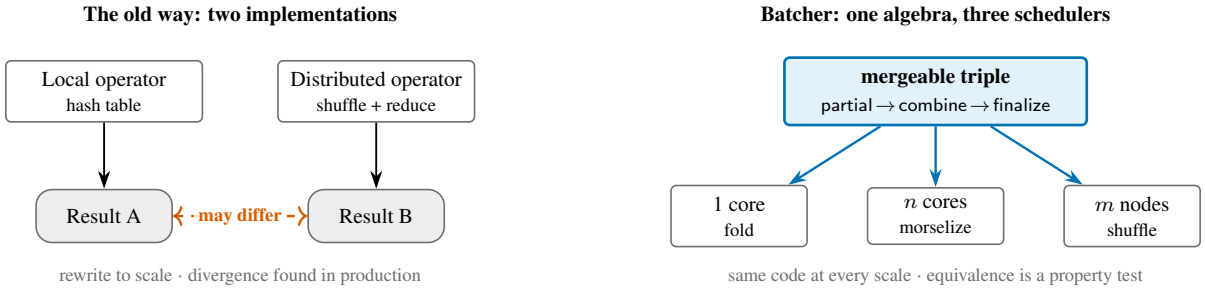

Figure 2: The decision the rest of the system is built on. Batcher admits an operator only if it can be written as a mergeable triple, a partial pass, an associative and commutative combine, and a finalize, and then varies only the *scheduling*. Aggregation, distinct, join build, window and top-N all qualify. No second implementation exists to disagree with the first, so scaling up is a configuration change rather than a migration.

model can tell you. But cost is not selectivity, and the case a static model cannot fix even in principle is two predicates of identical cost where only one is selective. Write the broad one first and nothing gets removed, so the selective one runs at full width anyway.

DuckDB solves this by hill-climbing over adjacent swaps. Batcher measures instead: because the short-circuit evaluates conjuncts one at a time, it can attribute rows and nanoseconds to each *individually* and order by time-per-row over fraction-removed, so the right order arrives after a **single morsel** where a hill-climb needs tens of batches to walk the permutation. On the adversarial shape, two regular-expression matches of identical static cost with the broad one written first, that is 48.64 ms down to 28.18 ms. Correctness costs nothing here, because the conjuncts of an `AND` commute and every order yields the identical mask.

Microseconds: the aggregate picks its own algorithm. A parallel aggregate normally hashes each morsel into its own table and merges the tables, which is right when grouping reduces and wrong when it does not: `GROUP BY l_orderkey, l_linenumber` over TPC-H `lineitem` reduces nothing at all, so every partial row survives and the merge inherits the whole relation. The alternative is to partition the input by key first and aggregate each partition exactly once. Measured on a 60M-row table across 96 cores, the two paths cross just under 0.18 rows kept per input row:

rows kept per input row	0.10	0.34	1.00
partial → combine (ms)	125.7	350.9	1351.6
partition → aggregate (ms)	198.2	185.3	370.2

This is more than a tuned threshold, for two reasons. The choice comes from a *measurement* and not an estimate: a distinct-count for a group key is a sketch, and after a filter it is a guess about a distribution nobody has looked at, so the executor partials a sample of morsels and reads the reduction those partials actually achieved, wasting no work either way. Second, the alternative path, partition → partial → finalize, is the exact composition Batcher runs *across machines*, executed here across cores. The high-cardinality single-node aggregate is the distributed algorithm, chosen at runtime. Refusing to fork the operator did not only prevent a class of bug; it handed the single-node executor a second strategy for free.

Stages and runs. The outer two clocks are the familiar ones. At a pipeline breaker the engine has already measured what it just processed, so it re-plans the remainder on real cardinalities. Across executions it accumulates sketches, refits cost coefficients and converges bandits. The three correct different errors, at timescales four orders of magnitude apart, and each reads only what has already been measured, so none of them is a prediction. The outer one is what makes a query cheaper the second time it runs, and none of the five comparators of Section 13 has it.

5 Pillar four: every shape of data, every tier of model

Shifts four and five: the data stopped being representable and the model stopped being downstream. These arrive together and this pillar answers them together, because they are the two axes of one grid: the *shape* of the data and the *tier* of the model. Both fragmented independently, and a team lands in the cross product: a job that reads Parquet and JPEGs, embeds one and joins the other, then scores the result with a gradient-boosted tree and an LLM in the same pass.

Engines specialize by *data shape*. DuckDB and Polars are superb over structured columns and have no image or audio surface at all. Ray Data and Daft handle multimodal data well, but Ray Data carries no relational optimizer and Daft's is rule-only, with no cost model to weigh a decode against the join above it. Spark covers structured and semistructured, and its multimodal story is a UDF. They specialize by *workload tier* too: classical machine learning in one framework, deep-model inference in another, LLM generation in a third, and none of them is the system holding the data. A team whose job spans both axes, which describes most teams, runs several systems and maintains the glue between them.

Batcher reads 30 file and table formats and exposes 125 public ML entry points, from generalized linear models through GPU inference to LLM batch generation. The size of those numbers matters less than the fact that a single plan spans them: Figure 3 is a four-by-four grid of data shape against model tier, and one optimizer covers all sixteen cells. The claim it supports is a bounded one, though: not that no engine can reach any given cell, but that none of the five comparators reaches all

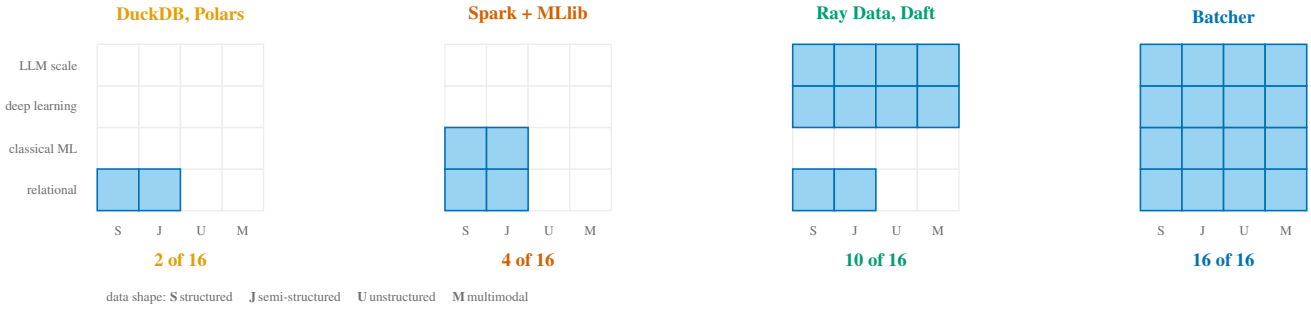

Figure 3: The second fragmentation, and the one that costs teams a second system. Engines specialize by *data shape* (columns) and by *workload tier* (rows), and the two axes fragment independently, so a job in the cross product needs more than one engine. A shaded cell means the group reaches that combination *natively, inside one optimized plan*, as characterized in this section; an unshaded cell is not a claim that the work is impossible there, only that it leaves the engine. Batcher is the one column that fills the grid, over 30 file and table formats and 125 public ML entry points. Shading is deliberately generous to the comparators: Daft's relational tier is credited to the Ray Data pair, and every group is given the widest reading its documentation supports.

sixteen *natively, in one plan, under one optimizer*.

In Batcher, preparing data for a model is not a different kind of work from querying it. Image decode, crop and normalization to a tensor are Arrow kernels in the data plane, not a Python UDF. So are audio decode, resampling, mel-spectrogram and MFCC, the last two matched to `torchaudio` within 10^{-6} and, across the comparator set of Section 13, without an in-engine equivalent anywhere else. So are vector distances, embedding pooling and fuzzy string matching. Vector search is a plain query. Batch inference runs against pools that load a model once per session and reuse it, with CPU decode and GPU compute overlapped so the accelerator is not idle while a JPEG is unpacked.

Epoch shuffling shows concretely what living inside the engine is worth. Permuting a training corpus normally means materializing an index of every sample, hundreds of gigabytes at ten billion of them, before the first batch is read. Batcher replaces the list with a keyed bijection, a small Feistel network [20] with cycle walking [4], so the order is computed, never stored, driver memory is constant in corpus size, and a mid-epoch resume is arithmetic [25].

So a retrieval pipeline, read a corpus, strip HTML, chunk, embed, write to a lakehouse table, is one plan and one pass over the data. So is a multimodal one: read images from object storage, decode and normalize, run a model, join the predictions against a customer table, then `MERGE` into Delta, with the filter that removes 90% of the rows running *before* the decode because one optimizer sees both. Others can do each of these things, and Daft in particular has a strong multimodal surface; what is unusual is that they are all expressions in one optimized plan over one columnar format, under one optimizer and one memory manager.

6 Pillar five: one scheduler across CPU and GPU

Shift six: compute stopped being homogeneous, so utilization became a question of overlap. Run a chain of unlike costs, I/O then CPU decode then a forward pass then a CPU join, inside one worker and the accelerator idles through every decode. This is the most common reason a GPU pipeline underperforms its hardware, and the usual remedy is for the user to hand-tune a ratio of CPU workers to GPU workers per stage.

Batcher treats it as a scheduling problem the engine owns. A plan carrying a resource-class boundary is split there: a pool of CPU producers decodes and preprocesses, a pool of GPU consumers runs the model, and Arrow batches move between them over the same credit-bounded Flight transport the shuffle uses, so the window between the stages is bounded and a slow consumer applies backpressure instead of accumulating. Both stages run the identical sub-plan through the ordinary executor and only the placement differs, which is what keeps the split from becoming a second semantics. A chain with no resource boundary to split at falls back to the embarrassingly-parallel map, so a homogeneous pipeline pays nothing for the mechanism.

Two smaller decisions carry more weight than they appear to. Models load once per *session* and are reused across executions, so a pipeline that runs the same model twice pays for it once; respawning per execution instead makes model load a per-query tax that dominates short jobs. And a GPU is divisible, so several small models can be packed onto one device by fractional allocation, none of them reserving a whole card. Section 13.6 reports what this is worth against a runtime purpose-built for GPU pipelines: 2–3.6×, with fractional-GPU packing at the bottom of that range. The GPU stages achieving those numbers are relational operators, under the same optimizer as the joins around them.

7 Pillar six: a Python surface the optimizer can see through

Shift seven: the interface moved to Python, so the escape hatch became the main road. The historical bargain was that SQL is restrictive and therefore optimizable, while a user-defined function is expressive and therefore opaque. That bargain fails when the practitioner writes Python by default: the natural way to express a transformation becomes an opaque per-row callback,

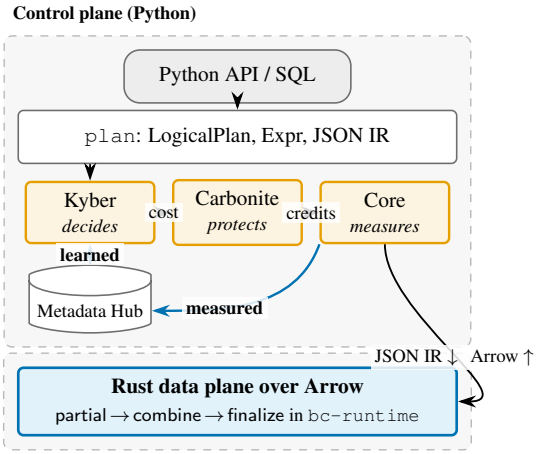

Figure 4: The contract loop. Python builds and optimizes a plan and ships it as JSON IR; all per-row work happens in Rust over Arrow and returns zero-copy. Core *measures*, Kyber *decides*, Carbonite *protects*, and the three cannot import one another, enforced on every commit. The highlighted path is the one the other engines do not have: what execution measured is what the next plan is built from, so a query gets cheaper the second time it runs.

which the optimizer cannot reorder, cannot push a predicate through, and cannot run anywhere but a Python interpreter.

Batcher's answer has three parts, in order of preference. First, make the UDF unnecessary. The public surface is a Python expression API (`col("x") * col("y")`, `.str.contains(...)`, `.list.cosine_similarity(...)`, `.image.to_tensor(...)`, `.audio.mel_spectrogram(...)`), and every one of those lowers to the same `bc_expr::Expr` the SQL front-end produces, so it is optimized, JIT-eligible and executed in Rust. Python builds the plan and never touches a row. That is what the breadth in Figure 3 is for: the transformations that would otherwise be UDFs are operators.

Second, when a callback is genuinely required, make it batch-first. A Batcher UDF receives a whole Arrow `RecordBatch` and returns one, so the interpreter is entered once per sixteen thousand rows, not once per row. A class-based UDF keeps its state, whether a loaded model, a tokenizer or a connection, across batches and across executions, which is what makes the session-warm pools of Section 6 possible at all.

Third, keep SQL. It remains the right language for a relational question, so the same engine accepts it through `bt.sql` and a session catalog, parses it to the same `LogicalPlan`, and puts it through the same 375 rules. Vector search shows the two surfaces meeting: `ORDER BY list_cosine_similarity(emb, [...]) LIMIT k` is an ordinary SQL query, and it runs the same kernel the Python accessor calls. Neither surface wraps the other, because both lower to one plan.

8 Architecture

Batcher is a Python control plane over a Rust data plane, with Arrow `RecordBatch` the only columnar contract at every internal boundary and across the FFI: roughly 100,000 lines of Rust across 13 crates and 233,000 of Python. Python builds a plan, optimizes it, and ships it as JSON; data returns zero-copy through the Arrow C Data Interface, and Python never iterates a row.

The three subsystems in Figure 4 are isolated by a lint contract that forbids them importing one another. The rule reads as bureaucratic until you see what it prevents: an optimizer pass that starts collecting its own telemetry, or an executor path that starts making optimization decisions, will compile and pass every test while silently corrupting the feedback loop the design depends on. Two execution tiers likewise share one expression type: an interpreter that is the correctness oracle, and a compiled fast path that must be bit-for-bit identical on the subset it supports and fall back silently otherwise. A divergence there, one tier wrapping an integer overflow where the other raises, is invisible in any single result, because which tier ran depends only on whether the expression happened to compile.

9 How Batcher compares

DuckDB, Daft and Spark set the bar Batcher is measured against, with Polars and Ray Data bracketing the space. **DuckDB** [26] is the single-node bar: vectorized, a compressed native store with zone maps, optimized once before execution, and it does not distribute. **Spark** [1, 35] distributes, and its Adaptive Query Execution [29] re-plans on measured statistics at stage boundaries, but its unit of movement is a row-shuffle over JVM serialization and its fixed per-query cost is large. **Ray Data** [32], on Ray [24], is a streaming runtime for ML pipelines with actor pools for GPU work and no relational optimizer. We benchmark against it throughout, because a data engine claiming to serve AI workloads should be held to the standard of the runtime those workloads use today; it is a yardstick rather than a rival. **Daft** [9] and **Polars** [34] are native Rust engines over Arrow [30], and Daft distributes over Ray.

Table 1 places Batcher against all of them, and the pattern in it is the paper's thesis in tabular form. Batcher does not lead because its kernels are faster than everyone else's; on several rows it is at parity, and on storage it is behind. It leads where the previous generation shipped a constant and Batcher ships a measurement, and where the previous generation required a second system.

What we took from Spark, and where we depart. Spark's authors got the hard parts right, and several of its ideas are still doing work here: treating a streaming query as a batch query over an unbounded table, re-planning once real cardinalities are known, and compiling inner loops instead of interpreting them. None of those were mistakes; they were right for the hardware of the time.

The sharpest departure is the unit of movement. Spark moves rows through JVM serialization, which was reasonable when no

	Batcher	DuckDB	Polars	Spark	Ray Data	Daft
Single-node / distributed	one algebra	single-node	single-node	two paths	two paths	two paths
Optimizer	375 rules, bushy DP	rules + DP	rules	Catalyst	n/a	rules
Cardinality from	sketches + measured	table stats	estimates	table stats	n/a	estimates
Cost model	refit from runtime	static	static	static	n/a	static
Physical strategy	bandit over past runs	heuristic	heuristic	heuristic	n/a	heuristic
Memory sizing	learned per-row	static	static	static	static	static
Re-optimizes mid-query	stage boundary	no	no	stage boundary (AQE)	n/a	no
In-engine audio / mel	native kernels	none	none	none	UDF	UDF
Vector search in SQL	yes	no	no	no	n/a	no
GPU batch inference	warm pools	none	none	external	actor pools	actor pools

Table 1: Batcher's competitive set is DuckDB, Daft and Spark; Polars and Ray Data bracket the space, the latter an adjacent streaming ML runtime and not a relational engine, so its relational rows read n/a. Blue marks a capability no other column has, amber its absence. The decisive block is the middle: every input to a planning decision derived from a measured run rather than shipped as a constant. No other engine here has that, and Section 2's single algebra is what makes it sound, by making a measurement taken on one core valid on ninety-six.

ecosystem agreed on a columnar format, and Arrow settled that. Batcher moves columnar batches with no serialization step: the memory one operator wrote is the memory the next one reads, and the boundary to Python is a pointer handoff. This is the largest single source of the gap in Table 2, and it is an ecosystem change more than an engineering one. The other two follow from shifts already argued. Spark's fixed per-query overhead buys fault tolerance and elasticity that matter above a certain size, but its local and distributed paths are the same path, so unlike Batcher it has no choice to make below that size. And Catalyst optimizes from catalog statistics with AQE correcting at shuffles, which Batcher keeps and brackets with a clock on either side (Section 4.1); the outer one has no Spark equivalent.

10 Kyber: an optimizer that learns

10.1 375 rules, and what makes them safe

Seven phases run in a fixed order. The rewrite phases iterate to a fixpoint because their rules are confluent; the cost-based and physical phases run once, because they make a decision and do not converge to one. Measured evidence enters at three of them: cardinality drives join ordering, refit cost coefficients drive fusion, and a bandit drives the physical selection.

A rule set this size is safe only if each rule preserves results and no two interfere, and Kyber proves both mechanically. Each rule carries a plan-shape unit test and a differential test against DuckDB across nulls, empties and the type edges, and the set as a whole is property-tested: a generator builds a random table and a random valid pipeline and asserts that the result under all 375 rules equals the result under none. The same suite proves confluence and termination, so the rules provably do not oscillate.

Provenance gates the dangerous rules. Every cardinality and column statistic carries *where it came from*, and the two rule families that can corrupt a result rather than merely slow one, dropping a `Distinct` over a column proven unique and folding a subtree proven empty, fire only on `EXACT` provenance. A sketch estimate may never drive either, because dropping a `Distinct` on a wrong guess returns quickly, with duplicate rows, and nothing turns red. Everything else may consume an estimate, because the worst outcome there is a slower plan.

Provenance says where a number came from and nothing about whether it was *right*, which is a boundary easy to cross by accident: the gate deciding whether a query pays for stage-by-stage re-optimization used the label as a proxy for "we do not know this size," so a shape estimated to within a percent of actual still read as a guess. It now asks the evidence instead, treating a shape as confidently sized when its recorded history of predicted-against-actual rows stayed inside the re-optimization threshold every time. Over the 22 TPC-H queries, **15 of 22 routed to staging on a cold hub, and 0 once their estimates had demonstrably held up**, while all 15 still route to staging when the recorded history shows the estimates missing.

10.2 Cardinality and cost, from four sources

Estimation starts from Selinger-style selectivities [28] and is superseded in fixed order by exact footer metadata, then by sketches built during execution (HyperLogLog [10] for distinct counts, KLL [15] for quantiles), then by measured cardinalities recorded under a plan signature. Every sketch is mergeable with a fixed seed, so one built per partition merges identically to one built on a single node: the algebra of Section 2 applied to statistics.

Getting this wrong is expensive in a specific way. TPC-H's `orders.o_orderkey` carries no distinct count in any file footer, and without one the join estimator assumes a primary-key relationship, which is right for a fact-to-dimension join and catastrophic for a many-to-many key. At sf10 that priced one join at 1.5 million rows where the true output is about six billion, and the executor set about materializing it, passing 110 GB resident. Kyber now measures the distinct count before the optimize that fixes join order for the whole query; seeded, the planner picks the order DuckDB picks, and that query falls from $8.81\times$ to $3.95\times$ of DuckDB. Cost itself collapses CPU, I/O and network to one scalar, with per-operator coefficients *recalibrated from measured operator times*, and join ordering is dynamic programming over *bushy* plans throughout.

10.3 Two learned primitives, and one testing obligation

All of Kyber's self-tuning reduces to two reusable pieces, both keyed by plan signature and scoped to the host's hardware profile, so a value learned on a 16-core laptop never steers a 96-core planner. A **deterministic UCB1 bandit** [2] selects over a fixed arm set from measured per-arm latencies, driving the join-strategy choice and the staged-versus-one-shot routing decision above; there is no RNG, because a nondeterministic optimizer makes every performance regression unfalsifiable. An **OLS two-line crossover** learns a *threshold* where the bandit learns a choice, solving for the build size at which the cheaper-below algorithm is overtaken. Both inherit the result-invariance of Section 4, and every read is best-effort: a missing or malformed key yields the shipped default and never an exception into planning.

What a learning loop has to test. A learned component can be entirely inert while every result stays correct, and that asymmetry is the one genuinely new hazard this design introduces. Three defects here had that shape, each producing right answers at every scale while the mechanism that makes plans improve did nothing at all; fixing one took an sf10 query's steady state from $\sim$340 ms to $\sim$165 ms. A learning system therefore needs assertions that *the learning happened*, because a correctness suite passes identically whether the loop is running or switched off.

11 Carbonite: resource decisions from measured footprints

Carbonite decides whether a plan is feasible, hands out memory reservations and shuffle credits, and decides when a query goes out of core. It never rewrites a plan and never computes a result.

11.1 Admission, and a memory model that is a ratio

Conventional admission control answers a boolean: does this plan fit? A rejection is useless to a planner, because it names no smaller thing to try. Carbonite returns the binding constraint *and* a counter-offer, a smaller credit window or a lower parallelism, that Kyber can re-plan around, and it names the *operator* that binds, not the resource that ran out. This follows from the subsystem split: because Carbonite may not rewrite the plan, the only useful thing it can return is enough information for the layer that may.

Sizing used to come from the plan estimate alone, never from what operators actually used, and two of the choices in fixing that are forced, not tuned. Fit a per-input-row *ratio* and not an absolute peak, because a family's absolute peak depends on query size while a bytes-per-row figure multiplies by this plan's estimated input rows; it is the same reason the cardinality layer learns selectivity and not a row count. And fit the 80th percentile and not the median, because memory sizing is one-sided: over-provisioning costs a little headroom, under-provisioning costs the process. A cold store is a pass-through, so a first run is byte-for-byte the pre-learning behavior.

11.2 Credits, spill, and transport

Flow control is credit-based: one credit is one in-flight batch slot, and a producer blocks at zero, so a channel's window is a direct bound on its memory. A recurring shuffle warm-starts from the window past runs converged on, and because learning moves only the starting point, a stale value costs a few round trips of convergence and never correctness.

One spill property deserves separating out, because nothing accounted for it. Published shuffle output is memory nobody was reserving: a mapper hands a bucket to the node's transport layer and it stays resident until a reducer collects it, so with as many mappers as reducers a node holds its whole share of the shuffle against no reservation. It is now reserved, and deliberately the pool's *first* spill victim, because a published bucket is finished work waiting to be collected.

Bulk data never enters the Ray object store; it moves over Arrow Flight [31], with a reducer's sources tiered automatically by locality. An eight-producer gather runs at 33.6 GB/s over shared memory against 4.5 GB/s over loopback, and a miss falls back transparently, so single-node equals distributed regardless.

12 Execution: six operator results and two proofs

The executor pulls small batches through linear runs of operators and materializes only at pipeline breakers, in the style of morsel-driven parallelism [18]. Six operator-level results came out of that model, each verified against the sequential oracle before it was timed: a short-circuiting conjunctive filter (1.5–5.7×), dictionary-native comparison (19.6×), a runtime filter sunk across joins (1.9×), a deterministic parallel join map (1.8×), a bounded join fan-out (18× less memory), and a real inequality-join operator (up to 900×). Individually they are unremarkable. Three points about them are not.

Two rewrites that are correct by construction. Most optimizations are justified by testing: apply the fast path, compare against the slow one, ship it when the tests pass. Two here rest on an argument instead, one short-circuiting a conjunctive filter by compacting rather than masking, the other comparing dictionary-encoded columns without decoding them. Both are bit-identical for reasons statable in a sentence, which is a stronger guarantee than a passing test suite and is what lets them live inside the correctness oracle instead of beside it.

The memory bound is asserted, not assumed. Every operator that preserves or reduces rows satisfies "one batch in, one batch out" for free, so Section 3's constant-memory bound is immediate for almost all of them. A join that *multiplies* rows is the exception: a skewed key can turn each probe row into thousands of output rows, so the operator bounds both its output

batches and the index arrays that describe them. On a query where the unbounded version peaked at 13.1 GB, that holds peak memory to 728 MB, an $18\times$ reduction that also cuts runtime threefold with identical results. A correctness suite cannot observe a memory property, so a memory guarantee that is not asserted is not tested.

Where the algebra charges rent. Inequality joins used to lower to a cartesian product with a filter above it, quadratic in time and memory however few rows survive. They are now a real operator implementing IEJoin [16]: a case that took 3.4 seconds at ten thousand rows now takes under four milliseconds, and cases that did not run at all now do. IEJoin is also more general than most inequality joins need, so a *band*, `r.y BETWEEN l.a AND l.b`, and with it interval containment, temporal overlap and range lookup, earns an operator of its own: constraining one key from both sides makes the matches a contiguous slice, and one merge replaces a search per row. Measured against DuckDB reading its own store and correctness-gated at every size, the ratio at 500 K/1 M/2 M/5 M rows per side moves from $0.69/0.98/1.29/2.24\times$ to $0.49/0.55/0.64/0.94\times$: a $2.2\times$ loss at the top end becomes parity, and the smaller sizes become 1.6–$2.0\times$ wins.

13 Evaluation

Setup. One Intel Xeon Platinum 8275CL at 3.00 GHz, 96 vCPU, 184 GiB, Linux 6.8; distributed results on an 8-node Ray cluster, 128 vCPU; GPU results on 8×T4. DuckDB 1.5.5, Polars 1.36.1, Daft 0.7.21, Ray 2.56.0, PySpark 4.2.0, PyArrow 19.0.1. The harness refuses to report a timing for any query whose result does not match DuckDB, compared as an order-independent multiset, and timings are best-of-N warm. Two measurement hazards deserve naming. A shared machine is not a benchmark machine: at load average 16–41 a repeated run swung $\pm 25\%$, so claims moving less than $2\times$ are quoted from a run at load under 5. And an in-place rebuild of the compiled extension silently replaces the engine mid-benchmark, so we stamp its timestamp around every run and discard any run in which it moved.

13.1 Does the algebra remove the fork?

The central claim is architectural, so its evidence is a testing argument. Equation (1) is property-tested over random partitionings; an integration test asserts single-node equals multi-partition; and an operator matrix crosses {collect, spill, iterate, distributed} with {nulls, empty, one row, duplicates, -0.0/NaN, descending}, because the defects that reach production here are combinations of a non-default flag with a non-default path.

13.2 TPC-H

Table 2 is the headline, and it splits cleanly. Against Polars and Daft, Batcher leads by 1.8–$2.2\times$ in geometric mean at both scales. Against DuckDB the result splits along the line Table 1 predicted: on identical Arrow input Batcher wins 21 of 22 at sf10 and is $1.89\times$ faster overall, and against DuckDB's compressed native store it loses. q1 and q6 are essentially scans and sit at $1.47\times/1.52\times$, about what reading compressed pages instead of raw Arrow buys, but q21 ($3.18\times$), q9 ($3.79\times$), q7 ($3.19\times$) and q5 ($3.95\times$) are several times that, and those are engine work.

Correctness is not free for everyone. Daft gets TPC-H q6 wrong at both scales, dropping the `l_discount = 0.07` rows because $0.06 + 0.01$ in IEEE double is 0.06999999999999999, and Polars makes the same mistake; ground truth computed independently in PyArrow equals the official answer. Hence the gate on every timing. A benchmark that does not check results reports the fastest wrong answer.

13.3 Operator mix and ClickBench

Table 3 is the cleanest comparison here: one input representation, one machine, no SQL front-end differences, no scan in the timed region for anybody. Batcher sweeps PyArrow's Acero engine on all seven cases it can express and takes 8 of 11 from DuckDB and 9 of 11 from Polars.

On ClickBench [6] (43 queries, correctness-gated) Batcher wins 43 of 43 against DuckDB on identical Arrow input, every query, by 0.00–$0.42\times$, and 35 of 40 against Polars (the five losses are sub-3 ms queries where fixed overhead dominates). Against DuckDB's *native* store it wins a minority: 15 clear wins, 9 within $\pm 10\%$, 19 clear losses. We report the split and not a win count because a fifth of the suite sits inside the noise a shared machine adds, and a run of ours on a quieter box put it at 26 of 43. The attribution survives that uncertainty: the same engine on the same 43 queries wins *all* of them on identical Arrow input and a minority against a compressed store with zone maps. One variable, isolated, and the residue is storage, not kernels.

13.4 The small case, which a warm benchmark never measures

Section 2's first consequence is that a small query pays no distribution tax, and a suite reporting best-of-N warm timings cannot check it: it never sees a shape for the first time, which is what a user issues constantly. Measured separately, with a fresh literal per iteration so the plan cache cannot hit, optimizing a never-seen single-table filter costs 1.52 ms and a filter → join → group-by 2.75 ms, against 0.002 ms for a repeated shape. End to end over twelve never-seen shapes at $n = 10^4$, Batcher runs at $0.80\times$ DuckDB on a cold filter and $0.27\times$ on a cold `count(*)` over one. DuckDB pays a first-query cost too, and ours is the smaller. This is where a Python control plane could have been disqualifying and is not: 375 rules over seven phases iterated to a fixpoint cost single-digit milliseconds because nothing in them is per-row.

13.5 Scaling out

Aggregate all-to-all shuffle throughput runs 2.0 GB/s at two worker nodes, 6.9 GB/s at four and 15.2 GB/s at eight: super-

Baseline	sf1 geomean b/x	sf1 wins	sf10 geomean b/x	sf10 wins	Note
DuckDB (native compressed store)	1.02×	12/22	1.49×	3/22	compressed store
DuckDB (same Arrow input)	—	—	**0.53×**	**21/22**	like-for-like*
Polars [34]	**0.55×**	19/22	**0.45×**	17/22	
Daft [9]	**0.48×**	18/20	**0.52×**	13/20	wrong answers on q6, q15†
Spark [1]	0.041×	22/22	0.074×	22/22	reads Parquet in timed region‡

Table 2: TPC-H, all 22 queries, correctness-gated, one 96-core node. b/x is Batcher$_{ms}/x_{ms}$: **below 1.00 means Batcher is faster.** Suite totals (ms), sf1 / sf10: Batcher 618 / 4,491; DuckDB 616 / 2,353; Polars 1,256 / 10,624; Daft 1,224 / 9,261; Spark 14,555 / 57,781. *Separate run: DuckDB executing the same zero-copy Arrow Batcher is given, not a compressed store it was allowed to build first; 4,453 vs 8,436 ms. †Daft covers the 20 queries it can express; Batcher agrees with DuckDB on 22 of 22 at both scales. ‡Spark reads Parquet inside the timed region where the other four receive an in-memory Arrow handle, so its ratio is overstated. Ray Data is evaluated in Section 13.6 instead.

Operator case	DuckDB	Polars	PyArrow	Daft
`groupby-sum`	0.80	0.42	0.60	0.37
`groupby-2key`	0.62	0.40	0.54	0.22
`global-sum`	0.04	0.06	0.04	0.02
`filter-count`	0.07	0.03	0.00	0.03
`join-agg`	1.25	1.18	0.32	0.46
`sort-limit`	1.09	0.02	0.01	0.10
`filter-project`	0.79	1.10	0.06	0.49
`window-rank`	0.84	0.12	—	hangs
`window-runsum`	0.50	0.15	—	hangs
`window-lag`	0.71	0.04	—	hangs
`window-sum-part`	0.64	0.85	—	hangs
Batcher wins	8/11	9/11	**7/7**	**7/7**

Table 3: Operator mix at sf1, Batcher$_{ms}/x_{ms}$; below 1.00 is a Batcher win. All engines take the same in-memory Arrow input, so this carries none of Table 2's scan asymmetry. PyArrow (Acero) has no window functions.

linear, 7.6× for 4× the hardware. The mechanism is the algebra. Mappers publish pre-aggregated state rather than rows, so a combiner tree plus credit-based flow control keeps per-node memory bounded as the cluster widens, and adding nodes adds reduce capacity instead of contention. A *single* reducer stays NIC-bound at ∼2.7 GB/s, so the gain is in the aggregate. The superlinearity deserves separating from the headline: at two nodes the shuffle does not saturate the available reduce capacity, so part of the gain from 2 to 4 is recovered inefficiency. The claim we stand behind is the shape, that adding nodes adds throughput without a per-node memory penalty, and not a promise of 1.9× per doubling forever.

The single-query picture matches. On TPC-H sf10 q6 with every engine reading the same S3 Parquet and correctness-gated, Batcher over 64 partitions runs 224.5 ms against Daft on its Ray runner at 535.8 ms and DuckDB-on-Arrow single-node at 457.3 ms, and Daft returns the wrong answer described above. Across a pipeline suite at sf1/sf10/sf100 Batcher is ahead of Ray Data on every pipeline and leads Daft on joins (1.7–2.2×), group-by and metadata-only counts.

13.6 GPU and ML workloads

These are the workloads Ray Data was built for, which makes it the right yardstick and the wrong thing to keep score against. The question is whether an engine that treats decode and inference as relational operators can hold its own against a runtime purpose-built for them. It can. On 8×T4 with real models: ResNet-50 batch inference 2.05×, image embeddings 1.98×, fractional-GPU packing 1.96×, video-clip inference 3.6×, training-data ingest 3.0×, multimodal JPEG-to-GPU 1.3–2×.

Three figures are much larger and should not be quoted as general execution speed: audio feature extraction 12.5×, LLM batch inference 11.1×, text embeddings 47×. They come from scheduling, not kernels. The session-scoped model pools of Section 5 amortize load across executions, so the ratio shrinks as jobs grow; and native in-engine preprocessing keeps mel-spectrogram and MFCC as Arrow kernels where the comparator runs a per-batch Python UDF. Neither is an architectural advantage, both are choices any runtime could adopt, and we expect the gap to close. The durable claim is the 2–3.6× range, which shows that putting AI operators inside a query engine costs nothing against a specialized runtime. One exception bounds it: on a maximally-large compute-bound job both engines saturate the same GPU at the same FLOPs, so the ratio is parity. Nothing here beats a specialized runtime at arithmetic, and nothing here claims to.

A note on the harness. These ratios are only meaningful if the comparator was configured properly, and ours initially was not: our adapter registered each table as a single block, which is Ray Data's unit of parallelism, so a 6M-row table ran on one core of 96. That was our defect and not a property of Ray Data. Every number above is from the corrected adapter, which reads tables back through the Parquet connector with row groups sized to Ray's own target block size.

14 Related work

Batcher's execution is morsel-driven [18] over a vectorized Arrow representation in the MonetDB/X100 lineage [5], with a compiled path for scalar expressions restricted to a subset carrying a proven-identical fallback. Volcano [11] estab-

lished the operator/exchange separation; the departure is that Batcher has no distributed operator to separate. Mid-query re-optimization [14], progressive optimization [23] and eddies [3] are the lineage of the stage-boundary clock, surveyed in [8], with Spark AQE [29] its closest deployed relative. Learned optimizers [21, 22] attack estimation with models, where Batcher takes the lighter route of sketches [10, 15], refit cost coefficients and a bandit [2] over result-equivalent strategies, buying reproducibility and a cold start that is exactly the untuned system. [19] is the standing argument for why measured cardinalities beat better estimators, and Structured Streaming's framing of a stream as an unbounded table [36] is the one Section 3 adopts.

15 Conclusion

The workarounds for the seven shifts are so familiar by now that they read as the cost of doing business. A warm cluster, so that small queries are bearable. A second execution model for anything fresher than a day. A shuffle width tuned by hand, and a CPU-to-GPU worker ratio picked the same way. A per-row callback for whatever SQL cannot say. And a duplicate of the data for whoever trains the models.

None of it is necessary, and one constraint retires those boundaries together. Admitting an operator only as a mergeable triple removes the split between the fast engine and the scalable one, so speed is not re-earned at scale and scale is not paid for in advance. Because that triple is also the incremental form, it removes the split between batch and streaming. Because the operator is identical everywhere, a measurement taken anywhere is valid everywhere, which is what lets the optimizer plan from evidence rather than from constants chosen years ago for hardware nobody is running. And because decode, embedding, vector search and inference are expressions in that same algebra over the same Arrow batches, the pipeline that turns raw bytes into model input is one optimized plan instead of three systems and two copies of the data. One algebra did all of it, and the breadth is not four features bolted together but one decision, taken early and never broken.

Reproducibility. Every number comes from a committed harness: `benchmarks/run.py` drives the engine adapters and `harness.py` enforces the correctness gate, refusing to time any query whose result does not match DuckDB. It records the engine build profile with each run, so a result cannot silently be attributed to a different build. Timings are best-of-N warm on the machine described in Section 13.

References

[1] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. Spark SQL: Relational data processing in Spark. In *ACM SIGMOD International Conference on Management of Data*, 2015.

[2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

[3] Ron Avnur and Joseph M. Hellerstein. Eddies: Continuously adaptive query processing. In *ACM SIGMOD International Conference on Management of Data*, 2000.

[4] John Black and Phillip Rogaway. Ciphers with arbitrary finite domains. In *Topics in Cryptology — CT-RSA*, 2002.

[5] Peter A. Boncz, Marcin Zukowski, and Niels Nes. MonetDB/X100: Hyper-pipelining query execution. In *Conference on Innovative Data Systems Research (CIDR)*, 2005.

[6] ClickHouse, Inc. ClickBench: A benchmark for analytical databases, 2024.

[7] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

[8] Amol Deshpande, Zachary Ives, and Vijayshankar Raman. Adaptive query processing. *Foundations and Trends in Databases*, 1(1):1–140, 2007.

[9] Eventual Computing. Daft: A distributed query engine for large-scale multimodal data, 2024.

[10] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms (AofA)*, 2007.

[11] Goetz Graefe. Volcano—an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.

[12] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data Mining and Knowledge Discovery*, 1(1):29–53, 1997.

[13] International Data Corporation. Worldwide global DataSphere structured and unstructured data forecast, 2025–2029. Technical Report US52800025, International Data Corporation, 2025.

[14] Navin Kabra and David J. DeWitt. Efficient mid-query re-optimization of sub-optimal query execution plans. In *ACM SIGMOD International Conference on Management of Data*, 1998.

[15] Zohar Karnin, Kevin Lang, and Edo Liberty. Optimal quantile approximation in streams. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2016.

[16] Zuhair Khayyat, William Lucia, Meghna Singh, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Nan Tang, and Panos Kalnis. Lightning fast and space efficient inequality joins. *Proceedings of the VLDB Endowment*, 8(13), 2015.

[17] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *Conference on Innovative Data Systems Research (CIDR)*, 2019.

[18] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. Morsel-driven parallelism: A NUMA-aware query evaluation framework for the many-core age. In *ACM SIGMOD International Conference on Management of Data*, 2014.

[19] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.

[20] Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM Journal on Computing*, 17(2):373–386, 1988.

[21] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. Bao: Making learned query optimization practical. In *ACM SIGMOD International Conference on Management of Data*, 2021.

[22] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. Neo: A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 2019.

[23] Volker Markl, Vijayshankar Raman, David Simmen, Guy Lohman, Hamid Pirahesh, and Miso Cilimdzic. Robust query processing through progressive optimization. In *ACM SIGMOD International Conference on Management of Data*, 2004.

[24] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[25] MosaicML / Databricks. StreamingDataset: Fast, accurate streaming of training data from cloud storage, 2023.

[26] Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *ACM SIGMOD International Conference on Management of Data*, 2019. Demonstration.

[27] David Reinsel, John Gantz, and John Rydning. The digitization of the world: From edge to core. IDC White Paper US44413318, International Data Corporation, November 2018.

[28] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *ACM SIGMOD International Conference on Management of Data*, 1979.

[29] The Apache Software Foundation. Adaptive query execution — Spark SQL performance tuning guide, 2024.

[30] The Apache Software Foundation. Apache Arrow: A cross-language development platform for in-memory analytics, 2024.

[31] The Apache Software Foundation. Arrow Flight RPC, 2024.

[32] The Ray Team. Ray data: Scalable datasets for ML, 2024.

[33] Transaction Processing Performance Council. TPC Benchmark H standard specification, 2022.

[34] Ritchie Vink and Polars contributors. Polars: DataFrames for the new era, 2024.

[35] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

[36] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2013.